

6.1 이미지 분류를 위한 신경망

- 입력 데이터로 이미지를 사용한 분류는 특정 대상이 영상 내에 존재하는지 여부를 판단하는 것이다.
- 이번 장에서는 이미지 분류에서 주로 사용되는 합성곱 신경망의 유형을 알아본다

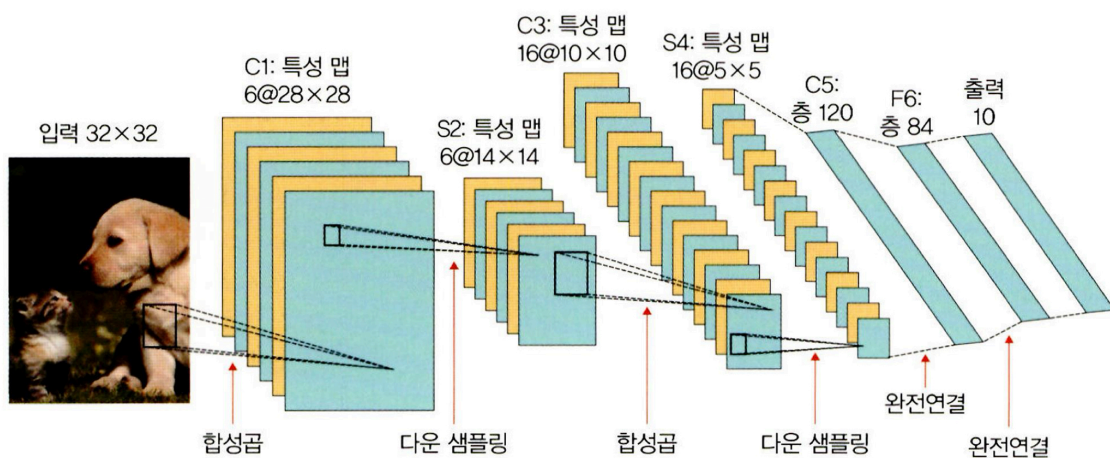
LeNet-5

합성곱 신경망이라는 개념을 최초로 얀 르쿤이 개발한 구조

1995년 얀 르쿤과 동료들이 수표에 쓴 손글씨 숫자를 인식하는 딥러닝 구조 LeNet-5를 발표했는데 그것이 현재 CNN의 초석이 되었다.

LeNet-5는 합성곱과 다운 샘플링을 반복적으로 거치며 마지막 완전연결층에서 분류를 수행한다.

▼ 그림 6-1 LeNet-5



LeNet-5의 과정 그림으로 살펴보기

C : 합성곱층, **S** : 풀링층, **F** : 완전연결층

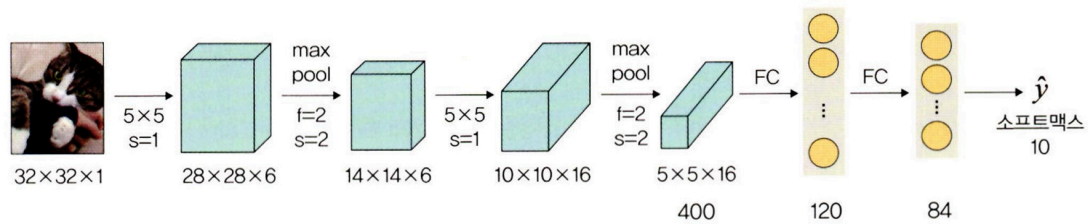
1. **C1**에서 5x5 합성곱 연산 후 28x28 크기의 특성 맵 6개를 생성한다.
2. **S2**에서 다운 샘플링하여 특성 맵 크기를 14x14로 줄인다.
3. **C3**에서 5x5 합성곱 연산하여 10x10 크기의 특성 맵 16개를 생성한다.
4. **S4**에서 다운 샘플링하여 특성 맵 크기를 5x5로 줄인다.

5. C5 에서 5x5 합성곱 연산하여 1x1 크기의 특성 맵 120개를 생성한다.
6. 마지막으로 F6 에서 완전연결층으로 C5의 결과를 유닛(또는 노드) 84개에 연결시킨다.

LeNet-5 예제 구현

- 구현할 신경망

▼ 그림 6-2 LeNet-5 예제 신경망



32x32 크기의 이미지에 합성곱층과 최대 풀링층이 쌍으로 두 번 적용된 후 완전연결층을 거쳐 이미지가 분류되는 신경망

▼ 표 6-1 LeNet-5 예제 신경망 상세

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	32x32	—	—	—
합성곱층	6	28x28	5x5	1	렐루(ReLU)
최대 풀링층	6	14x14	2x2	2	—
합성곱층	16	10x10	5x5	1	렐루(ReLU)
최대 풀링층	16	5x5	2x2	2	—
완전연결층	—	120	—	—	렐루(ReLU)
완전연결층	—	84	—	—	렐루(ReLU)
완전연결층	—	2	—	—	소프트맥스(softmax)

라이브러리 설치

- `tqdm` 라이브러리
 - 아나콘다 프롬프트에서 `pip(or conda) install` 명령어를 통해 설치한다.
 - 진행 상태를 가시화하여 보여주는 라이브러리
- `torchvision.transforms` : 이미지 변환(전처리) 기능 제공
- `torch.optim` : 경사하강법을 이용한 가중치를 구하기 위한 옵티마이저 라이브러리

- `os`: 파일 경로에 대한 함수 제공

1. 이미지 데이터셋 전처리

```
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize, scale=(0.5, 1.0)),
                transforms.RandomHorizontalFlip(),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]), # ---1.1
            'val': transforms.Compose([
                transforms.Resize(256),
                transforms.CenterCrop(resize),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ])
        }

    def __call__(self, img, phase): # ---1.2
        return self.data_transform[phase](img)
```

`transforms.Compose` : 다양한 이미지 변형 방식들을 한꺼번에 해주는 기능

`torchvision.transforms` : 토치비전(torchvision) 라이브러리를 이용하여 이미지 전처리를 손쉽게 한다.

`__call__` : 클래스 호출 메서드로, 클래스에 이 함수가 있을 경우 클래스 객체 자체를 호출하면 콜 함수의 return값이 반환된다.

2. 이미지 데이터셋을 불러온 후 훈련, 검증 테스트로 분리

```
cat_directory = '/content/drive/MyDrive/dogs-vs-cats/Cat'
dog_directory = '/content/drive/MyDrive/dogs-vs-cats/Dog'

cat_images_filepaths = sorted([os.path.join(cat_directory, f) for f in
                                os.listdir(cat_directory)])
dog_images_filepaths = sorted([os.path.join(dog_directory, f) for f in
```

```

os.listdir(dog_directory]))
images_filepaths = [*cat_images_filepaths, *dog_images_filepaths] # 개와
고양이 이미지들을 합쳐서 저장
correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is not
None]

random.seed(42)
random.shuffle(correct_images_filepaths)
#train_images_filepaths = correct_images_filepaths[:20000] #성능을 향상시
키고 싶다면 훈련 데이터셋을 늘려서 테스트해보세요
#val_images_filepaths = correct_images_filepaths[20000:-10] #훈련과 함께
검증도 늘려줘야 합니다
train_images_filepaths = correct_images_filepaths[:400] # 훈련용 400개 이
미지
val_images_filepaths = correct_images_filepaths[400:-10] # 검증용 92개 이
미지
test_images_filepaths = correct_images_filepaths[-10:] # 테스트용 10개 이
미지
print(len(train_images_filepaths), len(val_images_filepaths), len(test_images
_filepaths))

```

- `cat_images_filepaths = ...` : 고양이 이미지 데이터를 가져옴

```

cat_images_filepaths = sorted([os.path.join(cat_directory, f)
                                (a) (b)
                                for f in os.listdir(cat_directory)])
                                (c)

```

- `sorted` : 데이터를 정렬된 리스트로 만들어 반환
- `os.path.join` : 경로와 파일명을 결합하거나 분할된 경로를 하나로 합치고 싶을 때 사
용
- `os.listdir` : 지정한 디렉터리 내 모든 파일의 리스트를 반환
- `correct_images_filepaths = ...` : `images_filepaths`에서 이미지 파일들을 불러옴

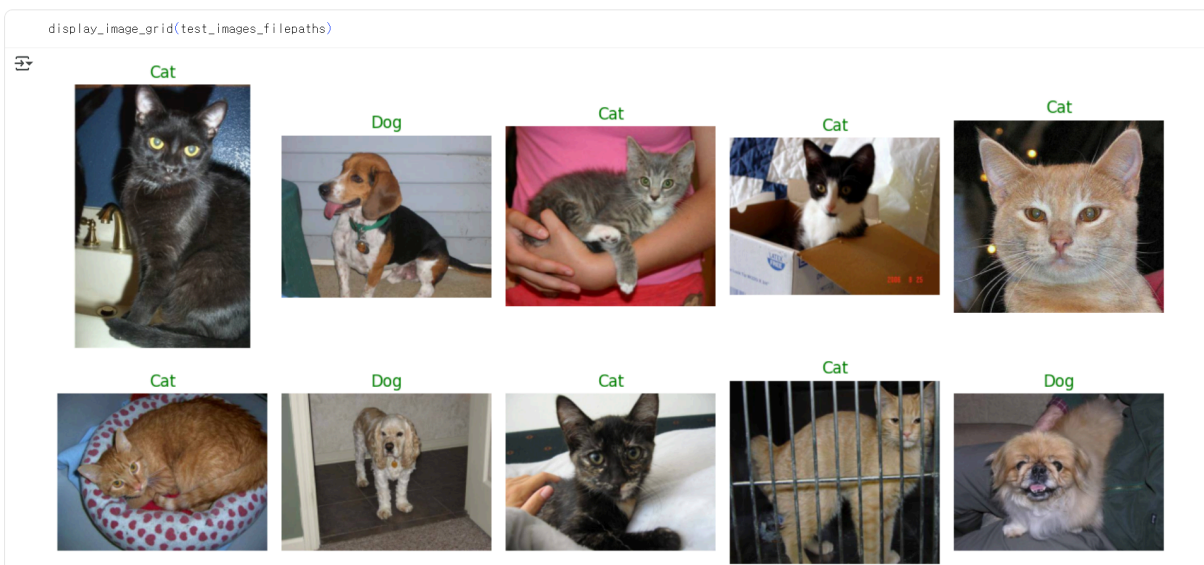

```
correct_images_filepaths = [i for i in images_filepaths
                             (a) (b)
                             if cv2.imread(i) is not None]
                             (c)
```

- `i` : for 반복문을 이용하여 가져온 데이터에 대해 `i`를 이용해 리스트로 만든다.
- `for i in _` : 반복문을 이용해 `images_filepaths`에서 이미지 데이터를 검색
- `if 조건문` : `cv2.imread()` 함수를 이용해 모든 데이터를 읽어옴

3. 테스트 데이터셋 이미지 확인 함수

```
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12, 6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        true_label = os.path.normpath(image_filepath).split(os.sep)[-2]
        predicted_label = predicted_labels[i] if predicted_labels else true_label
        color = "green" if true_label == predicted_label else "red" # 정답이면 gr
        een, 틀리면 red
        ax.ravel()[i].imshow(image) # 개별 이미지 출력
        ax.ravel()[i].set_title(predicted_label, color=color)
        ax.ravel()[i].set_axis_off() # 이미지 축 제거
    plt.tight_layout() # 이미지 여백 조정
    plt.show()
```

- `cv2.cvtColor(입력이미지, 색상)` : 이미지 색상 변경
- `os.path.normpath` : 경로명 정규화 (경로를 통일함)
- `split(os.sep)` : 경로를 / 혹은 \를 기준으로 분할



테스트용 이미지 출력

4. 데이터 불러오기 함수 정의

```
class DogvsCatDataset(Dataset):
    def __init__(self, file_list, transform=None, phase='train'): # 데이터셋 전처리
        self.file_list = file_list
        self.transform = transform
        self.phase = phase
    def __len__(self): # image_filepaths 데이터셋 전체 길이 반환
        return len(self.file_list)
    def __getitem__(self, idx):
        img_path = self.file_list[idx]
        img = Image.open(img_path) # img_path 위치에서 이미지 데이터를 가져온다.
        img_transformed = self.transform(img, self.phase) # 이미지에 'train' 전처리
        적용
        label = img_path.split('/')[-1].split('.')[0] # 이미지 데이터에 대한 레이블 값을
        가져옴
        if label == 'dog':
            label = 1
        elif label == 'cat':
            label = 0
        return img_transformed, label
```

- `img_path` : 이미지가 위치한 전체 경로 → 레이블을 가져오기 위해서 '/'와 '.'을 제거해야 함

- `img_path.split('/')[-1]` : 전체 경로에서 '/' 제거
- `split('.')[0]` : '.' 분리. 분리된 값들에서 첫 번째 값을 가져옴

5. 데이터로더 정의

```

▶ train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
  val_dataloader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
  dataloader_dixt = {'train': train_dataloader, 'val': val_dataloader}

  batch_iterator = iter(dataloader_dixt['train'])
  inputs, labels = next(batch_iterator)
  print(inputs.size())
  print(labels)

⇒ torch.Size([32, 3, 224, 224])
   tensor([0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 1,
           0, 0, 1, 1, 1, 0, 1, 1])

```

- `Dataloader` (데이터셋, batchsize, shuffle) : 배치 관리를 담당

6. 모델 네트워크 클래스 생성

```

class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.cnn1 = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=5, stride=1, padding=0)
        self.relu1 = nn.ReLU()
        self.maxpool1 = nn.MaxPool2d(kernel_size=2)
        self.cnn2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=5, stride=1, padding=0)
        self.relu2 = nn.ReLU()
        self.maxpool2 = nn.MaxPool2d(kernel_size=2)
        self.fc1 = nn.Linear(32 * 53 * 53, 512)
        self.relu5 = nn.ReLU()
        self.fc2 = nn.Linear(512, 2)
        self.output = nn.Softmax(dim=1)
    def forward(self, x):
        out = self.cnn1(x)
        out = self.relu1(out)
        out = self.maxpool1(out)

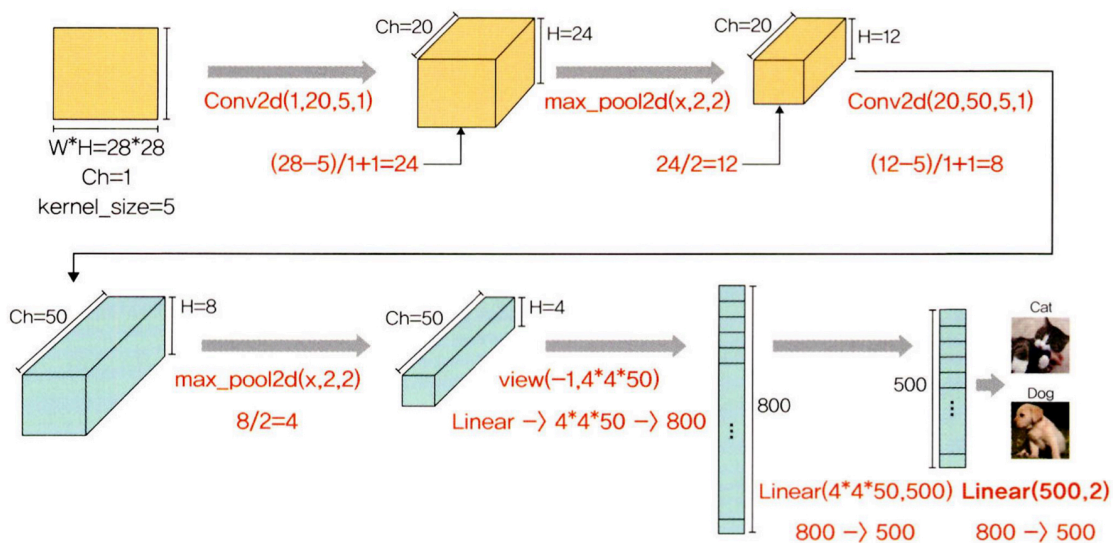
```

```

out = self.cnn2(out)
out = self.relu2(out)
out = self.maxpool2(out)
out = out.view(out.size(0), -1)
out = self.fc1(out)
out = self.fc2(out)
out = self.output(out)
return out

```

▼ 그림 6-5 합성곱층과 풀링층의 크기(형태) 계산



7. 옵티마이저와 손실함수 정의

```

optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
criterion = nn.CrossEntropyLoss()

```

- `optim.SGD()` : 모멘텀 SGD는 이전 수정 방향을 참고해 가중치를 수정한다.
 - `momentum` : SGD를 적절한 방향으로 가속화하며 진동을 줄여주는 매개변수

8. 모델 학습 함수

```

# 모델 학습 함수 정의
import time

```

```

def train_model (model, dataloader_dict, criterion, optimizer, num_epoch):
    since = time.time()
    best_acc = 0.0

    for epoch in range(num_epoch):
        print('Epoch {}/{}'.format(epoch+1, num_epoch))
        print('-'*20)

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()
            else:
                model.eval()
            epoch_loss = 0.0
            epoch_corrects = 0

            for inputs, labels in tqdm(dataloader_dict[phase]): # dataloader_dict: 훈련데이터셋
                inputs = inputs.to(device) # CPU에 할당
                labels = labels.to(device)
                optimizer.zero_grad() # 역전파 단계 전 기울기 초기화
                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels) # 손실 함수를 이용한 오차 계산
                    if phase == 'train':
                        loss.backward() # 모델의 학습 가능한 모든 파라미터에 대해 기울기 계산
                        optimizer.step() # 파라미터 갱신
                    epoch_loss += loss.item()*inputs.size(0)
                    epoch_corrects += torch.sum(preds == labels.data)
            epoch_loss = epoch_loss / len(dataloader_dict[phase].dataset)
            epoch_acc = epoch_corrects.double() / len(dataloader_dict[phase].dataset)
            print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_loss, epoch_acc))

            if phase == 'val' and epoch_acc > best_acc: # 최적 정확도 저장
                best_acc = epoch_acc
                best_model_wtss = model.state_dict()

```

```

time_elapsed = time.time() - since
print('Training complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_
elapsed % 60))
print('Best val Acc: {:.4f}'.format(best_acc))
return model

```



오차와 입력을 곱하는 이유?

```
epoch_loss += loss.item()*inputs.size(0)
```

손실함수 정의 시, reduction 파라미터를 지정할 수 있다.

reduction = 'mean'

'mean'은 정답-예측값의 오차를 구한 후 평균을 반환한다.

→ 전체 오차를 배치 크기로 나눠 평균을 반환하기 때문에, epoch_loss 계산하는 동안 loss.item()과 input.size(0)을 곱해주는 것!

```

# 모델 학습
import time
num_epoch = 10
model = train_model(model, dataloader_dict, criterion, optimizer, num_epoch)

```

9. 모델 테스트

```

# 모델 테스트 함수 정의
import pandas as pd

id_list = []
pred_list = []
_id = 0
with torch.no_grad():
    for test_path in tqdm(test_images_filepaths):
        img = Image.open(test_path)
        _id = test_path.split('/')[-1].split('.')[1]
        transform = ImageTransform(size, mean, std)
        img = transform(img, phase='val') # 테스트 데이터셋 전처리 적용
        img = img.unsqueeze(0)
        img = img.to(device)

```



```

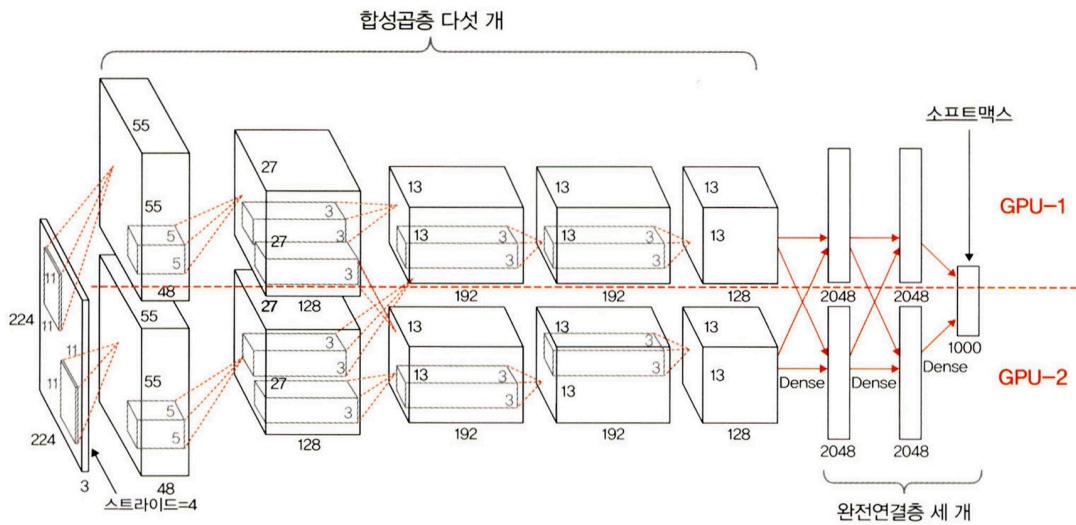
model.eval()
outputs = model(img)
preds = F.softmax(outputs, dim=1)[: , 1].tolist()
id_list.append(_id)
pred_list.append(preds[0])
res = pd.DataFrame({
    'id': id_list,
    'label': pred_list
})
res.sort_values(by='id', inplace=True)
res.reset_index(drop=True, inplace=True)
res.to_csv('LeNet', index=False)

```

- `torch.unsqueeze` : 텐서에 차원 추가, (0)은 차원이 추가될 위치를 의미
 - 1차원 형태의 텐서에 0위치에 차원을 추가하면 2차원이 됨.
 - 2차원 텐서에 0위치에 차원 추가하면 채널이 추가됨.
- `F.softmax(outputs, dim=1)[: , 1].tolist()`
 - `(outputs, dim=1)` : 아웃풋에 softmax를 적용해 각 행의 합이 1이 되도록 함.
 - `[: , 1]` : 모든 행에서 인덱스 1을 가져옴
 - `tolist()` : 배열을 리스트 형태로 변환

AlexNet

♥ 그림 6-9 AlexNet 구조



- GPU 두 개를 기반으로 한 병렬 구조인 점을 제외하면 LeNet-5와 유사함.

♥ 표 6-2 AlexNet 구조 상세

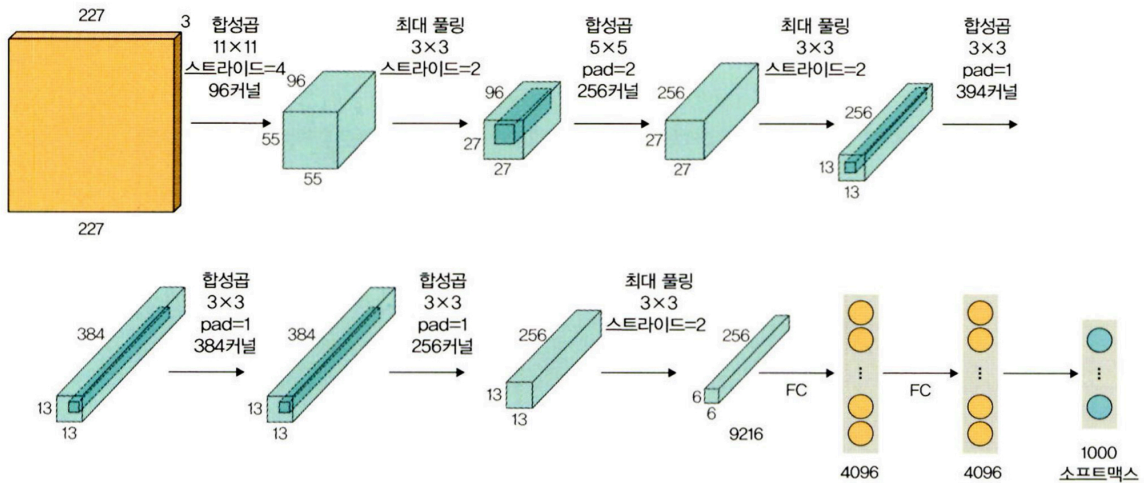
계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	227×227	—	—	—
합성곱층	96	55×55	11×11	4	렐루(ReLU)
최대 풀링층	96	27×27	3×3	2	—
합성곱층	256	27×27	5×5	1	렐루(ReLU)
최대 풀링층	256	13×13	3×3	2	—
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	256	13×13	3×3	1	렐루(ReLU)
최대 풀링층	256	6×6	3×3	2	—
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	1000	—	—	소프트맥스(softmax)

네트워크 입력 : 227x227x3 크기의 RGB 이미지

출력 : 1000x1 확률 벡터 (각 클래스에 해당)

AlexNet 예제 구현

♥ 그림 6-11 AlexNet 예제 네트워크



데이터 전처리 ~ 데이터 불러오기 까지는 위의 예제와 유사하기 때문에 설명 생략

AlexNet 모델 네트워크 정의

```
class AlexNet(nn.Module):
    def __init__(self) → None:
        super(AlexNet, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(64, 192, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(192, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
        )
        self.avgpool = nn.AdaptiveAvgPool2d((6, 6))
        self.classifier = nn.Sequential(
            nn.Dropout(),
```

```

nn.Linear(256 * 6 * 6, 4096),
nn.ReLU(inplace=True),
nn.Dropout(),
nn.Linear(4096, 512),
nn.ReLU(inplace=True),
nn.Linear(512, 2),
)

def forward(self, x: torch.Tensor) → torch.Tensor:
    x = self.features(x)
    x = self.avgpool(x)
    x = torch.flatten(x, 1)
    x = self.classifier(x)
    return x

```

- nn.ReLU(inplace=True) : 연산에 대한 결과값을 새로운 변수가 아니라 기존 데이터를 대체하여 저장
- nn.AdaptiveAvgPool2d((6, 6)) : 풀링에 사용하는 함수
 - 풀링에 대한 커널 및 스트라이드 크기를 정의해야 작동한다.
 - (N, C, H_in, W_in) → (N, C, H_out, W_out)

$$H_{out} = \left\lceil \frac{H_{in} + 2 \times \text{padding}[0] - \text{kernel_size}[0]}{\text{stride}[0]} + 1 \right\rceil$$

$$W_{out} = \left\lceil \frac{W_{in} + 2 \times \text{padding}[1] - \text{kernel_size}[1]}{\text{stride}[1]} + 1 \right\rceil$$

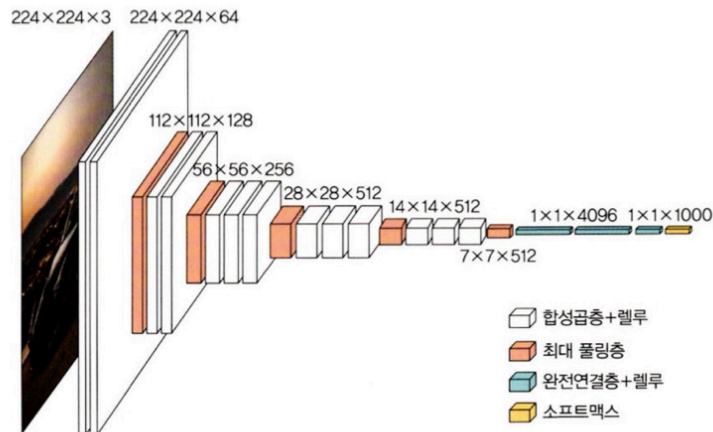
→ 그 이후의 과정도 LeNet-5와 동일하게 수행한다.

VGGNet

합성곱층의 파라미터 수를 줄이고 훈련 시간을 개선하기 위한 모델.

→ 네트워크를 깊게 만드는 것이 성능에 어떤 영향을 미치는지 확인하고자 함

▼ 그림 6-13 VGG16 구조



▼ 표 6-3 VGG16 구조 상세

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	224×224	—	—	—
합성곱층	64	224×224	3×3	1	렐루(ReLU)
합성곱층	64	224×224	3×3	1	렐루(ReLU)
최대 풀링층	64	112×112	2×2	2	—
합성곱층	128	112×112	3×3	1	렐루(ReLU)
합성곱층	128	112×112	3×3	1	렐루(ReLU)
최대 풀링층	128	56×56	2×2	2	—
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
최대 풀링층	256	28×28	2×2	2	—
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
최대 풀링층	512	14×14	2×2	2	—
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
최대 풀링층	512	7×7	2×2	2	—
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	1000	—	—	소프트맥스(softmax)

VGG11 구현 실습

- VGG11: VGGNet 중에서 가장 간단한 모델

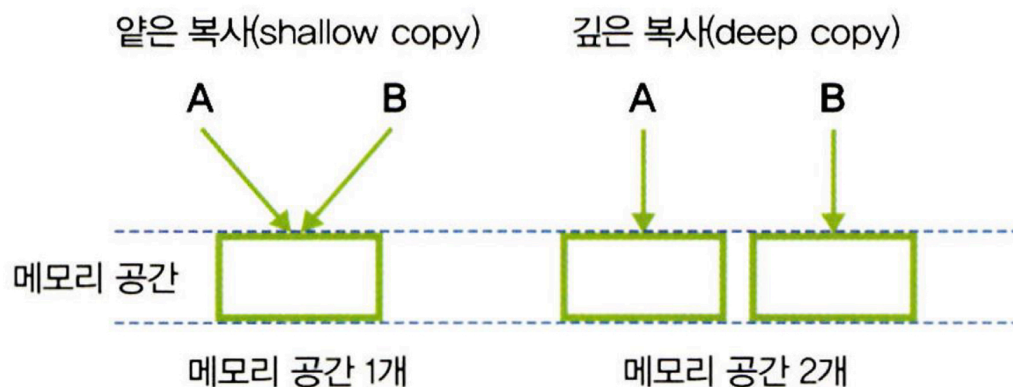
라이브러리 호출

```
import copy
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torch.utils.data as data
import torchvision
import torchvision.transforms as transforms
import torchvision.datasets as Datasets

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

- copy : 객체 복사를 위해 사용
 - 일반 복사 : 복사한 값과 원본 값 모두 바뀜
 - 얕은 복사 (copy.copy()): 복사값만 바뀜
 - 깊은 복사 (copy.deepcopy()): 복사값만 바뀌지만 메모리 수가 지시하는 변수의 수와 같음

▼ 그림 6-14 얕은 복사와 깊은 복사



VGG 모델 네트워크 정의

```

# VGG 정의
class VGG(nn.Module):
    def __init__(self, features, output_dim):
        super().__init__()
        self.features = features
        self.avgpool = nn.AdaptiveAvgPool2d(7)
        self.classifier = nn.Sequential(
            nn.Linear(512 * 7 * 7, 4096),
            nn.ReLU(inplace = True),
            nn.Dropout(0.5),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace = True),
            nn.Dropout(0.5),
            nn.Linear(4096, output_dim),
        )

    def forward(self, x):
        x = self.features(x)
        x = self.avgpool(x)
        h = x.view(x.shape[0], -1)
        x = self.classifier(h)
        return x, h

```

```

# 모델 유형 정의
vgg11_config = [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M']
# 합성곱층 8 + 풀링층 3 => VGG11

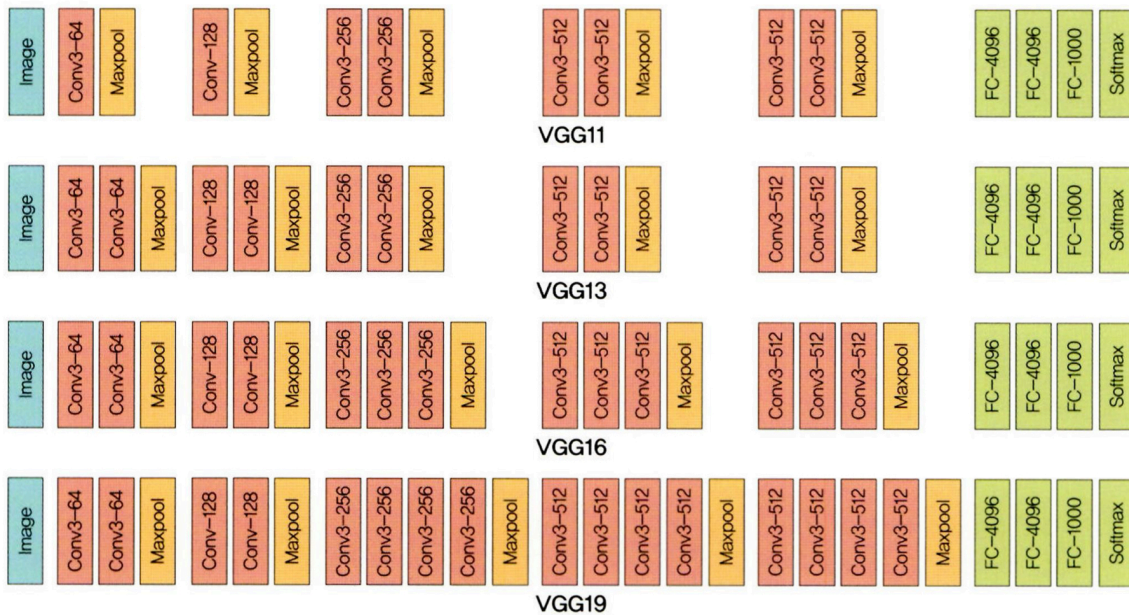
vgg13_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 'M', 512, 512, 'M', 512,
512, 'M']
# 합성곱층 10 + 풀링층 3 => VGG13
vgg16_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512,
'M', 512, 512,
512, 'M']
# 합성곱층 13 + 풀링층 3 => VGG16
vgg19_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512, 512,
512, 512, 'M',

```


512, 512, 512, 512, 'M']
 # 합성곱층 16 + 풀링층 3 => VGG19

- 숫자 : Conv2d 수행을 의미
- M : 최대 풀링 수행을 의미

▼ 그림 6-15 VGG의 다양한 모델에 대한 네트워크



VGG 계층 정의

```
def get_vgg_layers(config, batch_norm):
```

```
    layers = []
```

```
    in_channels = 3
```

```
    for c in config: # vgg11_config 값
```

```
        assert c == 'M' or isinstance(c, int)
```

```
        if c == 'M': # M은 최대 풀링 적용
```

```
            layers += [nn.MaxPool2d(kernel_size = 2)]
```

```
        else: # 숫자라면(M이 아니라면) 합성곱 적용
```

```
            conv2d = nn.Conv2d(in_channels, c, kernel_size = 3, padding = 1)
```

```
            if batch_norm: # 배치정규화+ReLU
```

```
                layers += [conv2d, nn.BatchNorm2d(c), nn.ReLU(inplace = True)]
```

```
            else: # only ReLU
```

```
                layers += [conv2d, nn.ReLU(inplace = True)]
```

```
            in_channels = c
```

```
return nn.Sequential(*layers)
```

- `assert c == 'M' or isinstance(c, int)` : c가 'M'이 아니거나 int가 아니면 오류 발생
 - `assert` : 가정 설정문, 뒤의 조건이 True가 아니면 오류 발생
 - `isinstance` : 주어진 조건이 True인지 판단

모델 계층 생성

```
vgg11_layers = get_vgg_layers(vgg11_config, batch_norm = True)
```

- `batch_norm` : 데이터 평균0, 표준편차 1로 정규화

사전 훈련된 VGG11을 사용하는 법

- 새로 네트워크를 구현하는 것이 아닌, 제공된 모델을 이용하는 법

```
import torchvision.models as models
pretrained_model = models.vgg11_bn(pretrained = True)
print(pretrained_model)
```

- `vgg11_bn` : VGG11 기본 모델 + 배치 정규화 적용된 모델을 사용
- `pretrained=True` : 사전 훈련된 모델을 의미

GoogLeNet

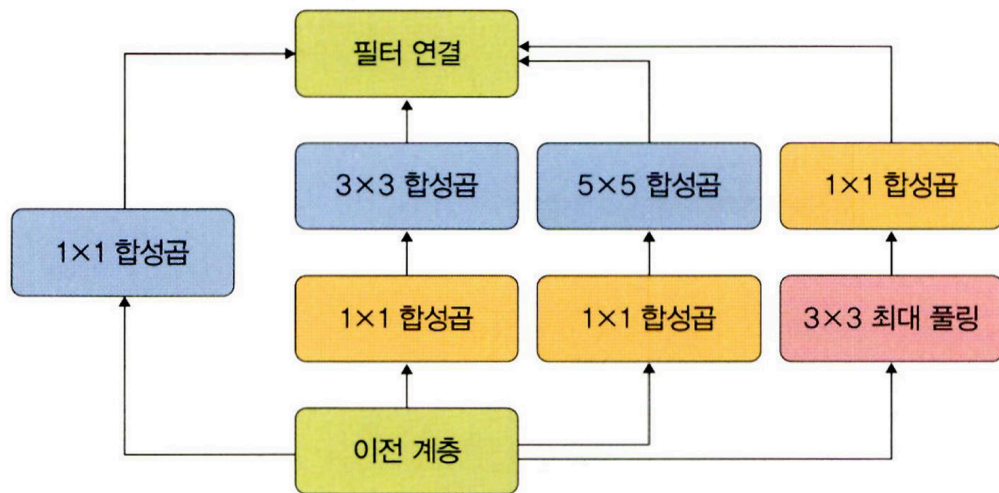
- 주어진 하드웨어 자원을 최대한 효율적으로 이용하면서 학습 능력은 극대화할 수 있는
깊고 넓은 신경망
- google + lenet



Inception 모듈

특징을 **효율적으로** 추출하기 위해 1x1, 3x3, 5x5의 합성곱 연산을 각각 수행함.
→ GoogLeNet에 적용된 해결 방법은 **희소 연결**

▼ 그림 6-23 GoogLeNet의 인셉션 모듈



- **희소 연결** : 밀집하게 신경망들이 연결된 CNN과 달리, 관련성이 높은 노드끼리만 연결하는 방법 → 적은 연산량, 과적합 해결